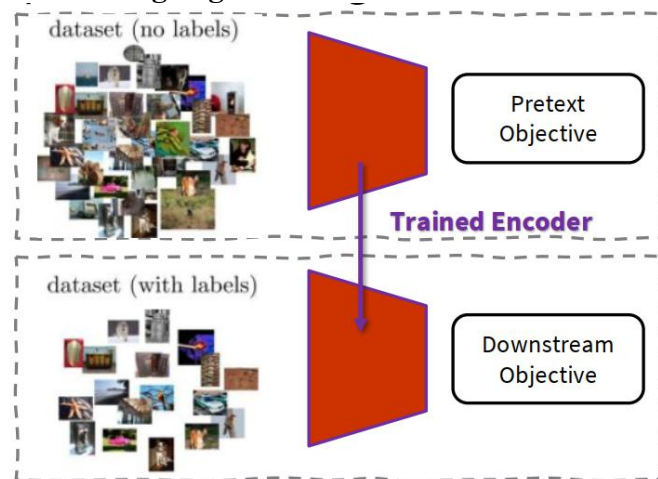


Problem: Học feature/representation cần rất nhiều data đã đánh label

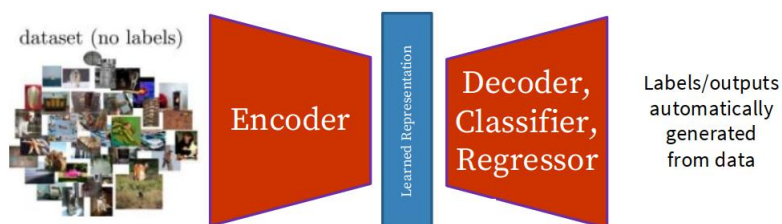
1. Self-supervised Learning là gì?



Self-supervised learning = học từ dữ liệu không nhãn bằng cách tự tạo mục tiêu huấn luyện từ chính dữ liệu, để học representation hữu ích cho task thật

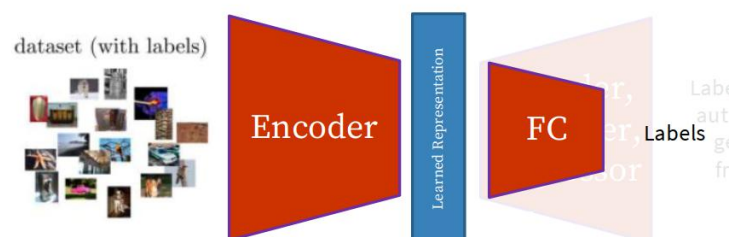
1.1. Pretext task

Là task được tạo từ data chính, không cần đánh label, có thể được coi là unsupervised learning task nhưng vẫn học bằng objective kiểu supervised như classification hoặc regression



1.2. Downstream objective

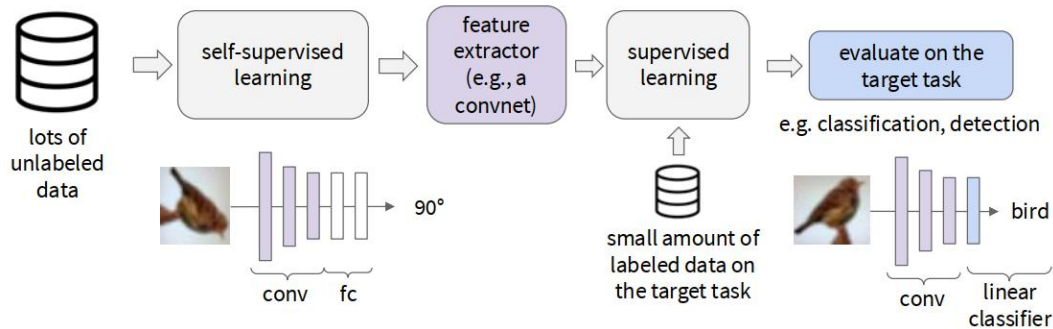
Là task chính (classification, detection, segmentation,...), nhưng kích thước dataset không lớn nhưng có nhãn, tận dụng encoder đã học từ pretext task



2. Cách đánh giá một phương pháp self-supervised

- Xem pretext task làm tốt đến đâu -> Chưa chắc là mục tiêu cuối (pretext tốt chưa chắc downstream tốt)
- Represent Quality: quan trọng, có các cách
 - + Linear evaluation protocol: freeze encoder, train một linear classifier lên representation.
 - + Clustering: xem representation có tự nhóm tốt không.
 - + t-SNE visualization: trực quan hóa xem feature có tách lớp tốt không
- Robustness và generalization: có ổn định khi sử dụng dataset khác không
- Tính toán có tối ưu không?

- Transfer learning và downstream performance: đánh giá SSL bằng cách xem feature nó học được có chuyển sang hỗ trợ tốt cho bài toán thật có nhãn hay không.



Note: SSL không chỉ dùng cho Computer Vision mà còn có thể xài cho nhiều bài toán khác

Today's lecture

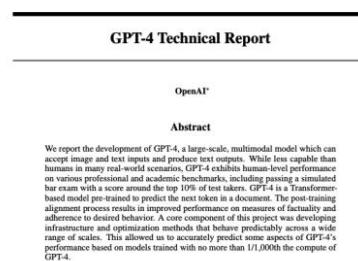


robot / reinforcement learning



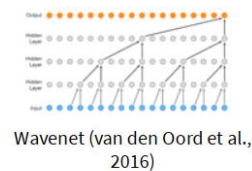
Dense Object Net (Florence and Manuelli et al., 2018)

language modeling



GPT-4 (OpenAI 2023)

speech synthesis

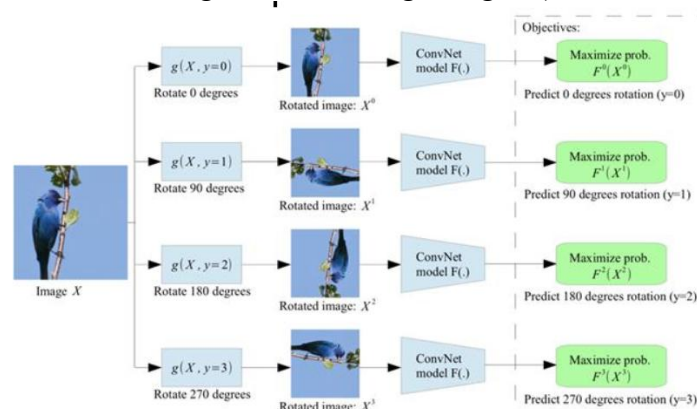


Wavenet (van den Oord et al., 2016)

3. Pretext task từ biến đổi ảnh

3.1. Rotation

Model sẽ thấy được ảnh quay các góc (0, 90, 180, 270) -> dự đoán quay góc nào
Hypothesis: Muốn biết ảnh nào là “đúng chiều”, model cần có một dạng visual commonsense (vd ếch ko treo ngược, chim đứng thẳng, ...)



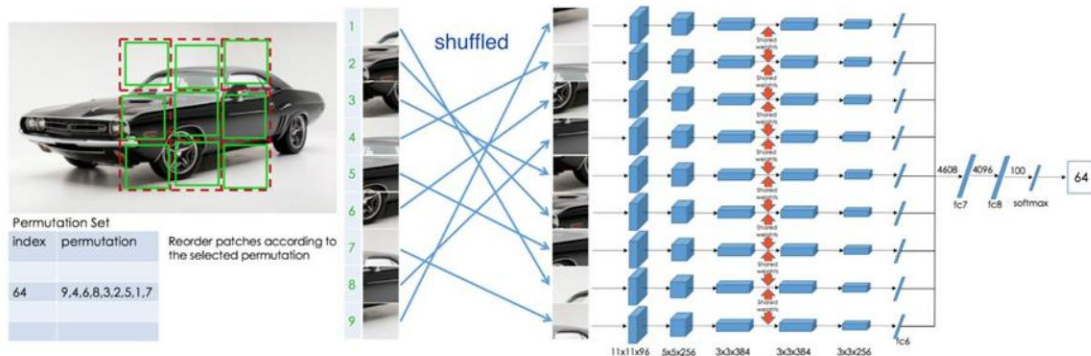
Note: trong attention map thì SSL với rotation có thể tập trung vào các đặc điểm chính giống hoặc hơn supervised model

3.2. Relative patch location

Chọn một patch trung tâm và một patch lân cận, model phải dự đoán patch thứ hai nằm ở vị trí tương đối nào so với patch trung tâm -> model này sẽ học kiểu part này sẽ phải nằm ở đâu trong object

3.3. Jigsaw puzzle

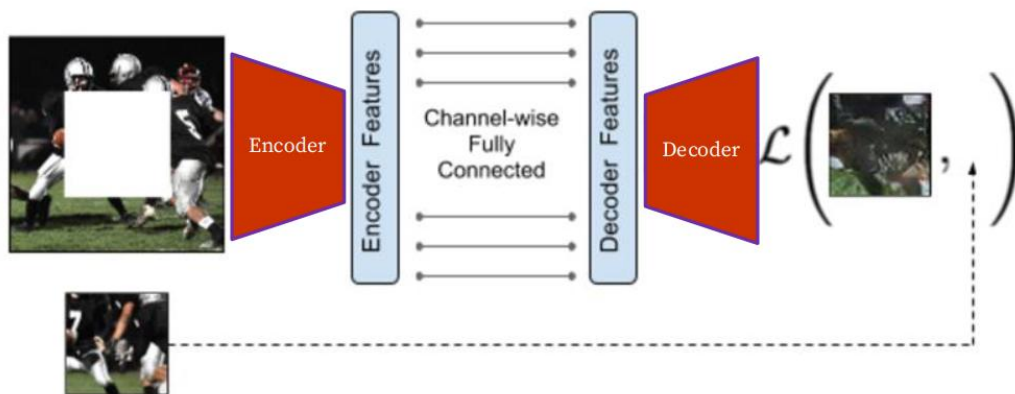
Cắt ảnh thành lưới patch rồi shuffle, model phải nhận ra cách hoán vị đúng hoặc suy ra cách sắp lại



Ví dụ có 9 patch -> 9! cách hoán vị -> model predict ra được cách hoán vị 64
-> cấu trúc toàn cục, quan hệ giữa các part và bố cục semantic

3.4. Inpainting

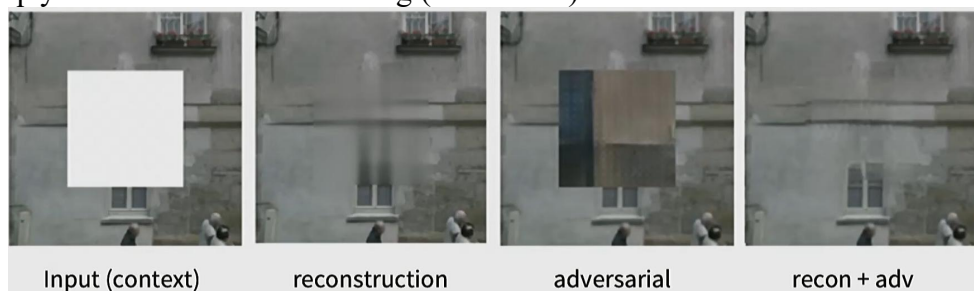
Che 1 phần giữa ảnh, model tái tạo lại phần bị che đó



Learning to reconstruct the missing pixels

Encoder học các background xung quanh -> decoder predict -> hàm loss sẽ bắt decoder phải sinh ra gần giống mức có thể

Nhược điểm: có thể paint theo nhiều hướng (có thể dẫn tới blur,...) -> hướng giải quyết thêm adversarial learning (lecture sau)



Input (context)

reconstruction

adversarial

recon + adv

Loss = reconstruction + adversarial learning

$$L(x) = L_{recon}(x) + L_{adv}(x)$$

$$L_{recon}(x) = ||M * (x - F_{\theta}((1 - M) * x))||_2^2$$

$$L_{adv} = \max_D \mathbb{E}[\log(D(x))] + \log(1 - D(F((1 - M) * x)))$$

Element wise multiplication

$$M = \begin{cases} 0 & \text{not masked} \\ 1 & \text{masked} \end{cases}$$

Encoder

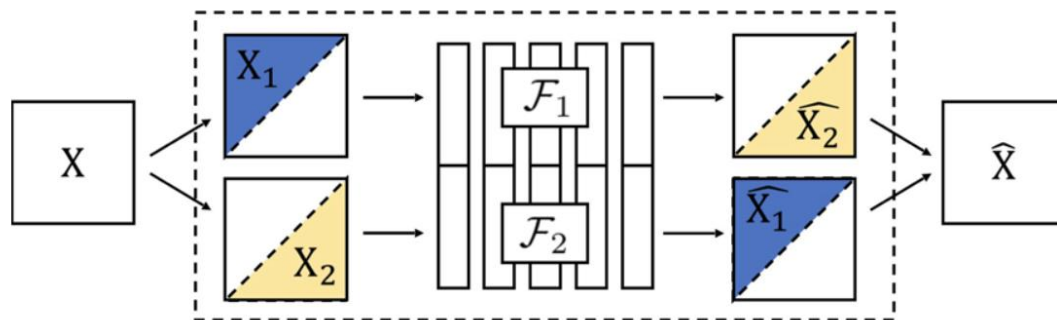
Giải thích qua Loss của recon thì $(1-M)*x$ là ảnh đầu vào sau khi che -> nhân F là qua bước encoder ra ảnh predict -> lấy x trừ đi để lấy loss -> xong nhân với M để chỉ tập trung vào phần bị mask

3.5. Image coloring

Tô màu ảnh trắng đen. Muốn tô màu đúng, model phải hiểu semantic (vd: trời xanh, lá cây xanh lá, da người có các tone màu nào,...)

*Split-brain autoencoder: dùng một phần kênh màu để dự đoán phần còn lại và dự đoán chéo giữa các channel

Idea: cross-channel predictions



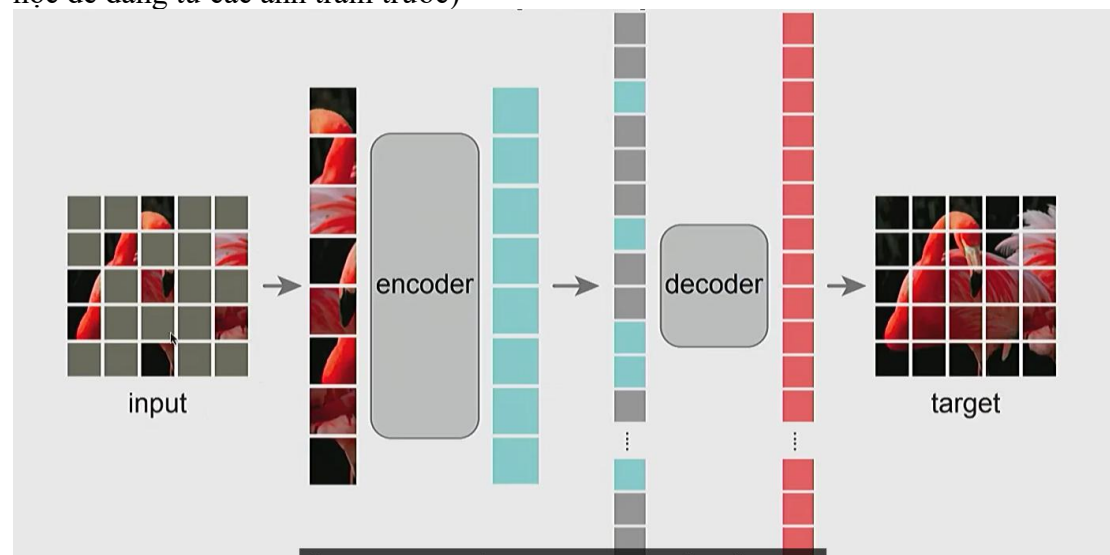
3.6. Video coloring

Có một reference frame có màu ở thời điểm $t=0$, còn các frame sau là xám. Model phải suy ra màu cho các frame tiếp theo

4. Masked Autoencoders (MAE) 1 framework thường dùng cho pretext task

Thay vì chỉ che một vùng nhỏ như inpainting cũ, MAE chia thành nhiều patch -> mask 1 tỉ lệ lớn (khoảng 50% - 75%) -> decoder cố gắng phục hồi

Masking làm task khó hơn và bắt model phải học kĩ các feature hơn (nếu mask ít thì học dễ dàng từ các ảnh train trước)



Encoder chỉ xử lý các patch thấy được không ẩn (linear projection và positional embedding) sử dụng transformer block

Decoder ghép output encoder với mask tokens ở vị trí bị che, thêm positional encodings, dùng transformer decoder rồi linear projection để reconstruct pixel

Asymmetric autoencoder: encoder mạnh, dùng cho downstream sau này, decoder chỉ để reconstruction khi pretraining, thường bỏ đi sau training -> giữ lại encoder vì nó phải học thông tin rất kĩ từ các patch ko bị che -> phù hợp cho các mô hình sau này
Loss: dùng MSE trong loss pixel, chỉ tính trên patch bị che giống inpainting

- * Linear Probing (đo chất lượng feature): Encoder (pretrained) -> giữ nguyên (freeze) -> Chỉ thêm 1 layer tuyến tính (Linear layer) ở cuối -> Chỉ train layer này
- * Full Fine-tuning: mọi thứ đều ko bị free -> train thêm cho task khác

5. Contrastive Representation Learning

CRL muốn representation của cùng object hoặc cùng sample thì gần nhau, khác object thì xa nhau

x	reference
x^+	positive
x^-	negative

Reference: ảnh gốc

Positive: chung object

Negative: khác object

Mục tiêu của CRL: $score(f(x), f(x^+)) \gg score(f(x), f(x^-))$

Các kiểu loss:

5.1. InfoNCE loss

$$L = -\mathbb{E}_X \left[\log \frac{\overbrace{\exp(s(f(x), f(x^+)))}^{\text{score for the positive pair}}}{\underbrace{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))}_{\text{score for the N-1 negative pairs}}} \right]$$

Giống cross entropy -log (nếu positive lớn thì loss nhỏ và ngược lại)

Ex là để lấy trung bình trên các mẫu (lấy mean trên các sample)

Bên cạnh đó nó còn đo mutual information (đo mức độ 2 biến chứa thông tin giống nhau)

A lower bound on the mutual information between $f(x)$ and $f(x^+)$

$$MI[f(x), f(x^+)] - \log(N) \geq -L$$

The larger the negative sample size (N), the tighter the bound

Mong muốn là muốn MI cao -> công thức này cho ta biết cận dưới của MI là

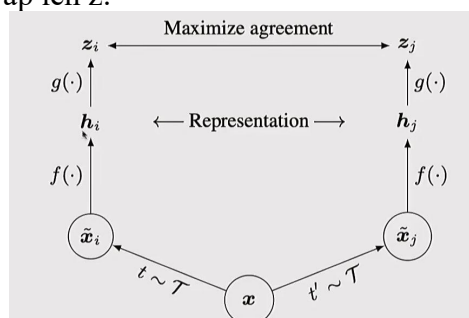
$\log(N) - L$ -> N lớn và L nhỏ thì MI sẽ cao hơn

Negative sample size(N) càng lớn -> cận dưới sẽ gần MI nhất giúp phản ánh MI tốt hơn. Vì sao N càng lớn thì càng tốt vì lúc đó model đã phân biệt đc khá tốt (vd 1 đúng 99 sai thì model phải cực tốt ms predict được)

5.2. SimCLR

SimCLR tạo positive pairs bằng data augmentation của cùng một ảnh: cropping, distortion, blur

Dùng thêm một network $g(\cdot)$ sau encoder để chiếu features sang không gian nơi contrastive learning diễn ra. Tức là: encoder tạo representation h -> projection head tạo z -> contrastive loss áp lên z .

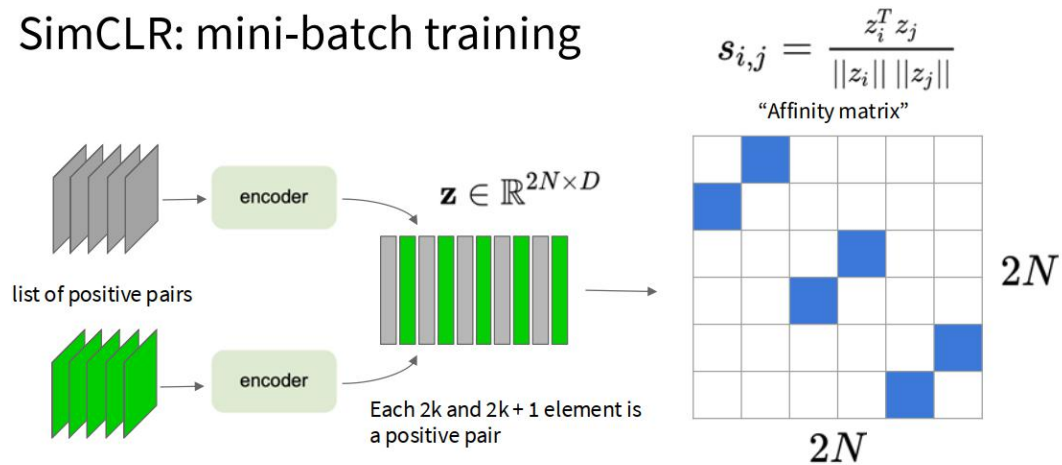


Project head giúp giữ lại nhiều thông tin hơn

Hàm mục tiêu sử dụng cosine similarity

* Batch construction: từ N ảnh gốc, augment mỗi ảnh thành 2 views -> được tổng cộng 2N samples, mỗi cặp views của cùng ảnh là 1 positive pair, tất cả sample còn lại trong batch là negatives -> SimCLR tận dụng in-batch negatives.

SimCLR: mini-batch training



*We use a slightly different formulation in the assignment.
You should follow the assignment instructions.

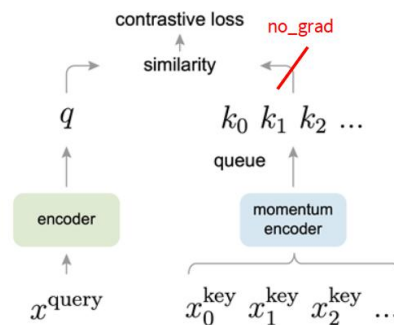
Màu xanh trong matrix -> giống nhau (positive pair)

* Training objective: Duyệt qua từng sample làm reference, positive là view còn lại của cùng ảnh,

negatives là các sample không phải positive, lấy trung bình loss trên toàn batch
Điểm yếu của SimCLR cần batch size lớn, batch lớn làm memory footprint rất lớn

5.3. Momentum Contrastive Learning (MoCo)

Problem: Trong SimCLR mỗi sample cần rất nhiều negative -> negatives thường lấy từ chính mini-batch -> nên muốn nhiều negative thì phải dùng batch rất lớn



x^{query} là ảnh đầu vào/input còn bên phải là các key được đẩy vào sau mỗi patch theo hàng đợi (vd mỗi batch vào khoảng 100 key) -> thường sẽ có 1 key là positive và sẽ được đánh dấu

Không cập nhật trực tiếp momentum encoder -> chỉ cập nhật encoder

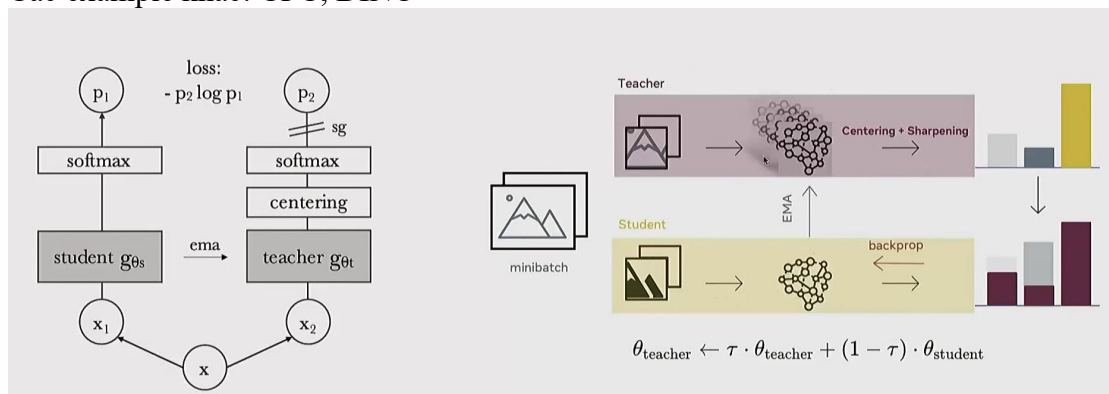
Cách cập nhật momentum encoder thông qua encoder

$$\bar{\theta}_k \leftarrow m\bar{\theta}_k + (1 - m)\theta_q$$

m là hệ số momentum (thường gần 1) k là tham số của momentum và q là của encoder -> momentum encoder sẽ là 1 phiên bản cập nhật của encoder và mượt hơn encoder -> giống DINO

Tại sao lại cập nhật chậm cho momentum: key cũ trong queue được sinh ra bởi encoder cũ ,key mới sinh ra bởi encoder mới -> feature space bị lệch mạnh -> queue sẽ kém ổn định

Các example khác: CPC, DINO



Ví dụ của DINO gần giống MoCo